

mkEngage Advanced Technology and Architecture Directive

This section overrides any conflicting technology, architecture, database, infrastructure, real-time communication, AI orchestration, analytics, or deployment instructions elsewhere in this prompt.

Build mkEngage using the most appropriate technology for each workload. Do not force the entire platform into one framework or programming language.

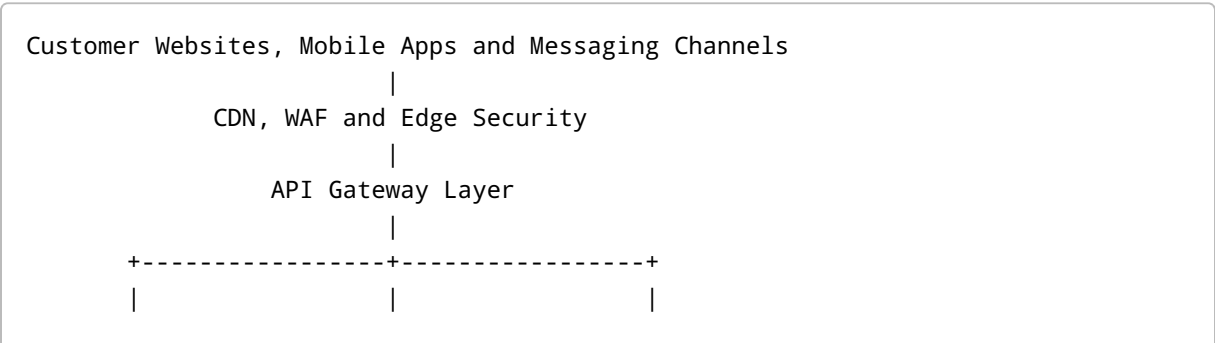
The architecture must support:

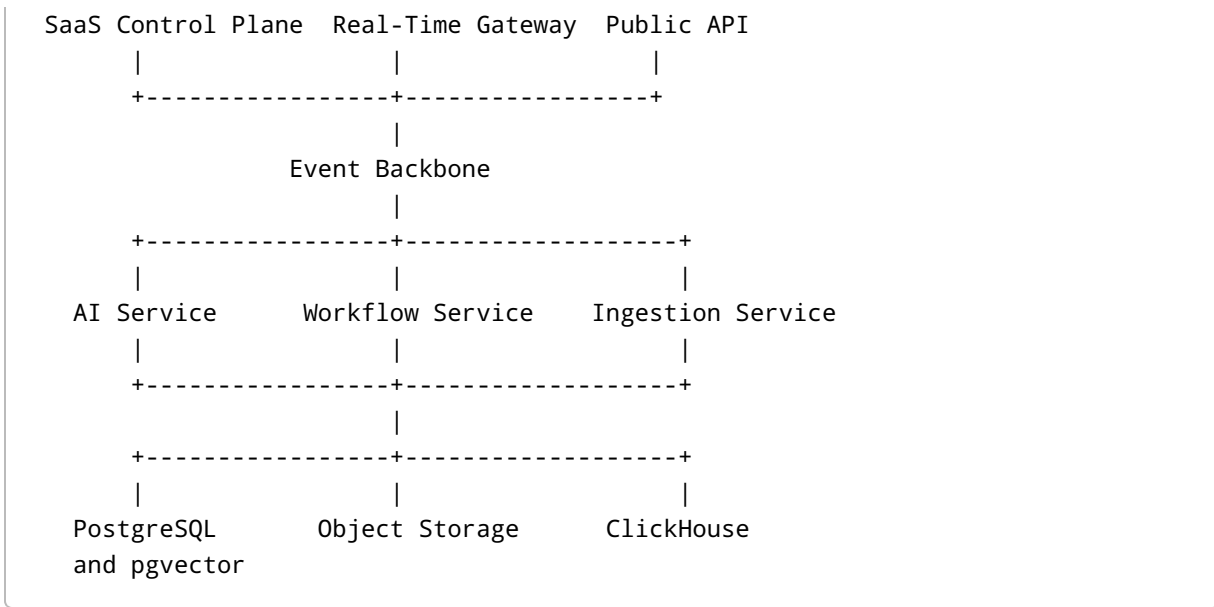
- Large numbers of concurrent chatbot connections
- Real-time bidirectional communication
- Multi-tenant SaaS isolation
- Omnichannel conversations
- AI knowledge retrieval
- Agentic AI actions
- Long-running workflows
- Human approvals
- High-volume visitor events
- Real-time analytics
- Horizontal scaling
- Regional deployment
- White-label environments
- Enterprise security
- Web, mobile, WordPress, Shopify, PHP, and Laravel integrations

Use a modular platform architecture with a small number of clearly separated services. Do not create unnecessary microservices or a distributed monolith.

1. Mandatory architecture strategy

Use the following application model:





Start with a modular control-plane application and independently deploy only the services that require different scaling characteristics:

1. SaaS control plane
2. Real-time messaging gateway
3. AI and RAG service
4. Durable workflow workers
5. Knowledge-ingestion workers
6. Analytics ingestion and querying

Do not split every feature into a separate service.

2. SaaS control plane

Use:

- Latest stable Laravel
- Latest stable supported PHP
- Strict PHP typing
- Laravel Octane
- FrankenPHP or RoadRunner
- Laravel Horizon
- Laravel Sanctum
- Laravel Fortify
- Composer
- Pest
- PHPStan at the highest practical strictness
- Laravel Pint

The SaaS control plane is responsible for:

- Authentication
- Organization onboarding
- Multi-tenancy
- Users
- Roles and permissions
- Departments
- Agent management
- Subscription plans
- Billing
- Feature flags
- White-label configuration
- Widget configuration
- Chatbot configuration
- Knowledge-source administration
- Integration administration
- API-key administration
- Audit logs
- Consent records
- Data-retention configuration
- Super-administrator functions
- Organization administration
- REST API management

Do not use Laravel as the primary high-concurrency WebSocket server.

Do not execute heavy document parsing, embedding generation, AI inference, analytical aggregation, or long-running workflows inside synchronous Laravel requests.

Laravel must dispatch those workloads to the appropriate service.

3. Administration dashboard

Use:

- Latest stable Next.js
- React
- TypeScript with strict mode
- Next.js App Router
- Server Components where appropriate
- TanStack Query
- Zod
- Tailwind CSS
- Radix UI primitives
- shadcn/ui components where appropriate

- React Hook Form
- TanStack Table
- React Flow for the workflow builder
- ECharts or Apache Superset embedding for advanced analytics
- Vitest
- React Testing Library
- Playwright

The dashboard must support:

- Server-side authorization
- Organization-aware routing
- Responsive design
- Light and dark modes
- Internationalization
- Right-to-left languages
- Accessible components
- Keyboard navigation
- WCAG 2.2 AA targets
- Optimistic updates where safe
- Real-time conversation updates
- Offline and reconnect states
- Consistent loading, empty, success, and error states

Use React Flow for:

- mkEngage Flow
- Chatbot flow editing
- Workflow visualizations
- Node configuration
- Branching
- Approval steps
- Integration actions
- Execution inspection

The Next.js App Router supports modern React functionality including Server Components, Suspense, and server-side functions. Use these capabilities where they improve security and performance.

4. Universal website chatbot widget

Build the widget as a standalone TypeScript package using:

- Lit
- Web Components
- Shadow DOM
- TypeScript strict mode
- Vite library mode

- Rollup-compatible production output
- IndexedDB
- Web Crypto API
- WebSocket client
- Server-Sent Events fallback where appropriate
- REST fallback
- i18next or an equivalent lightweight localization system
- Vitest
- Playwright
- Axe accessibility testing

Do not build the universal widget as a React application.

The widget must:

- Work on PHP websites
- Work on Laravel websites
- Work on WordPress
- Work on WooCommerce
- Work on Shopify
- Work on React
- Work on Next.js
- Work on Vue
- Work on Angular
- Work on Webflow
- Work on static HTML websites
- Avoid host-page CSS conflicts
- Load asynchronously
- Support code splitting
- Minimize initial bundle size
- Survive temporary network loss
- Restore conversation state
- Handle reconnection
- Handle message deduplication
- Support consent-aware visitor tracking
- Support signed authenticated identity
- Support keyboard and screen-reader navigation

Use Shadow DOM for style isolation.

Do not store secret API credentials in the widget.

5. Real-time messaging gateway

Use:

- Latest stable Elixir
- Latest stable Phoenix
- Phoenix Channels
- Phoenix Presence
- Phoenix PubSub
- ExUnit
- Distributed clustering supported by the deployment platform

The real-time gateway is responsible for:

- WebSocket connections
- Channel authorization
- Visitor presence
- Agent presence
- Typing indicators
- Message fan-out
- Delivery receipts
- Read receipts
- Connection recovery
- Heartbeats
- Conversation subscriptions
- Supervisor monitoring
- Agent collision indicators
- Live visitor updates

Every socket must be authorized using:

- Short-lived signed tokens
- Organization context
- Conversation access
- Role and permission checks
- Allowed-origin validation
- Token revocation checks

Use a per-conversation monotonically increasing sequence number.

Every message must include:

- Globally unique message ID
- Organization ID
- Conversation ID
- Channel ID
- Sender type
- Sender ID

- Sequence number
- Timestamp
- Idempotency key
- Correlation ID

The gateway must handle:

- Duplicate delivery
- Out-of-order messages
- Reconnect replay
- Expired sessions
- Agent disconnects
- Visitor disconnects
- Backpressure
- Rate limiting
- Slow clients
- Unauthorized subscriptions

Do not use WebSockets as the system of record. Persist accepted messages before confirming durable delivery.

6. AI and RAG service

Use:

- Latest stable Python
- FastAPI
- Pydantic
- Uvicorn
- Structured concurrency
- Async provider clients
- LangGraph for stateful agent execution
- Official AI-provider SDKs
- Pytest
- Ruff
- MyPy or Pyright
- OpenTelemetry instrumentation

The AI service is responsible for:

- Model routing
- Prompt templates
- Prompt versioning
- RAG
- Embeddings
- Hybrid search
- Reranking

- Citations
- Confidence scoring
- Intent detection
- Sentiment detection
- Conversation summarization
- Suggested agent replies
- Translation
- Knowledge-gap detection
- Tool selection
- AI safety enforcement
- Model fallback
- Token accounting
- Cost estimation
- AI evaluation

Support:

- OpenAI
- Azure OpenAI
- Anthropic
- Google AI
- OpenAI-compatible private models
- Customer-provided model credentials
- Private enterprise endpoints

Create a provider abstraction. Business logic must not import provider-specific implementations directly.

Provider configuration must support:

- Primary provider
- Primary model
- Fallback provider
- Fallback model
- Embedding provider
- Embedding model
- Reranking model
- Timeout
- Retry limit
- Monthly budget
- Per-conversation budget
- Maximum input size
- Maximum output size
- Data-retention preference
- Regional endpoint
- Allowed tools

Use LangGraph only for AI agent state and tool-selection orchestration.

Do not use LangGraph as a replacement for the core business workflow engine.

7. Durable workflow engine

Use Temporal for:

- Long-running chatbot workflows
- Human approval processes
- Refund approvals
- Order cancellation approvals
- Appointment workflows
- CRM synchronization
- Webhook retry workflows
- Integration synchronization
- Scheduled knowledge ingestion
- Data export
- Data deletion
- Subscription lifecycle operations
- AI tool executions
- Escalation timers
- SLA timers
- Agent handoff
- Multi-step external actions

Temporal workflows must be deterministic.

Place external activity in Temporal activities, including:

- Database calls
- Network calls
- AI requests
- Emails
- SMS
- CRM calls
- Shopify calls
- WordPress calls
- File processing
- Webhook delivery

Use:

- Explicit retry policies
- Exponential backoff
- Timeouts
- Heartbeats
- Cancellation handling

- Compensation steps
- Idempotency keys
- Versioned workflows
- Search attributes
- Correlation IDs

Do not implement long-running workflows using only database status columns and scheduled polling.

Do not use basic queues as a substitute for durable workflow state.

8. Event backbone

Use NATS with JetStream for operational event messaging.

Use it for events such as:

- Conversation created
- Message accepted
- Message delivered
- Message read
- Agent assigned
- Conversation transferred
- Visitor identified
- Visitor event recorded
- Knowledge source updated
- Knowledge ingestion completed
- Workflow started
- Workflow completed
- Tool approval requested
- Tool execution completed
- Integration failed
- Subscription changed
- Usage limit reached

Use:

- Persistent streams
- Durable consumers
- Explicit acknowledgements
- Dead-letter handling
- Replay
- Consumer backpressure
- Subject-based routing
- Message deduplication
- Schema validation

Represent internal events using:

- CloudEvents-compatible envelopes
- JSON Schema for payload contracts
- Semantic event versioning

Every event must include:

- Event ID
- Event type
- Event version
- Organization ID
- Actor ID when applicable
- Correlation ID
- Causation ID
- Timestamp
- Source service
- Payload

Use the transactional outbox pattern when publishing database-backed events.

Use inbox deduplication in consuming services.

Never depend on exactly-once delivery. Design consumers to be idempotent.

9. Primary transactional database

Use:

- Latest stable PostgreSQL
- UUIDv7 or ULID identifiers
- PostgreSQL Row-Level Security
- Native JSONB where flexible metadata is justified
- Table partitioning for very large event and message tables
- Connection pooling using PgBouncer or an equivalent managed pooler
- Point-in-time recovery
- Encrypted backups

PostgreSQL is the system of record for:

- Organizations
- Users
- Roles
- Permissions
- Contacts
- Visitors

- Conversations
- Messages
- Chatbots
- Workflows
- Knowledge metadata
- Integrations
- Subscriptions
- Feature flags
- API keys
- Webhooks
- Audit metadata
- Consent
- Approvals
- Tool executions

Enforce tenant isolation using both:

1. Application-level organization scopes
2. PostgreSQL Row-Level Security

Application scopes must not be the only tenant-isolation control.

Use transaction-scoped tenant context.

Create automated tests proving that PostgreSQL rejects unauthorized cross-tenant access.

10. Vector and hybrid knowledge search

Use PostgreSQL with pgvector for the initial and medium-scale vector-search implementation.

Implement hybrid retrieval using:

- pgvector semantic similarity
- PostgreSQL full-text search
- Metadata filtering
- Tenant filtering
- Permission filtering
- Document status filtering
- Freshness filtering
- Reciprocal Rank Fusion
- Optional reranking

Use HNSW indexes where appropriate.

Every knowledge chunk must include:

- Organization ID
- Source ID
- Document ID
- Document version
- Chunk ID
- Embedding
- Content
- Search vector
- Source URL
- Source title
- Access scope
- Publication status
- Valid-from date
- Expiration date
- Content checksum

Do not add a separate vector database during the initial implementation unless benchmark results prove PostgreSQL cannot satisfy the defined workload.

Create an adapter that allows migration to a dedicated vector platform later without rewriting the AI service.

11. Search technology

Use PostgreSQL full-text search for initial product search and knowledge retrieval.

Add OpenSearch only when required for:

- Large multi-field search
- Advanced faceting
- Very high document volume
- Complex log exploration
- Search workloads that exceed PostgreSQL benchmarks

Do not introduce OpenSearch only because it is listed in the architecture.

Document the benchmark or functional reason before enabling it.

12. Cache, sessions, rate limits, and ephemeral state

Use Redis for:

- Application cache
- Distributed rate limits
- Session support
- Temporary conversation state
- Presence metadata
- Agent capacity counters
- Distributed locks where unavoidable
- Frequently accessed configuration
- Short-lived identity tokens
- Idempotency caches

Redis must not be the permanent system of record for:

- Messages
- Conversations
- Approvals
- Billing
- Audit logs
- Knowledge
- Tool executions

Configure:

- Key namespacing by environment and organization
- Expiration policies
- Memory limits
- Eviction policies
- High-availability deployment
- TLS
- Authentication
- Monitoring

13. Real-time analytics

Use ClickHouse for:

- Visitor events
- Page views
- Session events
- Widget interactions
- Conversation metrics
- Agent performance

- Workflow execution metrics
- AI usage
- Token usage
- AI latency
- Tool execution metrics
- Campaign attribution
- Funnel analytics
- Cart recovery analytics
- Product-interest analytics
- SLA analytics

Stream analytical events through NATS JetStream into ClickHouse.

Do not run high-volume analytical queries against the primary transactional PostgreSQL database.

Create:

- Materialized views
- Aggregated tables
- Retention policies
- Organization-aware query controls
- Data freshness monitoring
- Backfill procedures

The mkEngage Insights interface must query ClickHouse for high-volume analytics.

PostgreSQL may be used for smaller administrative reports.

14. Object and file storage

Use S3-compatible object storage for:

- Uploaded documents
- Chat attachments
- Export files
- Generated reports
- WordPress packages
- SDK artifacts
- Archived conversation files
- Knowledge-source snapshots

Support:

- AWS S3
- Azure Blob through an adapter
- Google Cloud Storage through an adapter

- MinIO for local development

Use:

- Pre-signed upload URLs
- Pre-signed download URLs
- Short expiration
- Tenant-specific paths
- Content-type validation
- File-size restrictions
- Checksum validation
- Malware scanning
- Quarantine state
- Encryption
- Lifecycle rules
- Retention policies

Never expose storage credentials to clients.

15. API strategy

Use REST APIs with OpenAPI 3.1 as the primary external API contract.

Use:

- JSON
- Versioned endpoints
- Cursor pagination
- Idempotency keys
- Problem Details-compatible errors
- Request correlation IDs
- Rate-limit headers
- OAuth 2.1-compatible authorization
- Scoped API keys
- Webhook signatures

Create AsyncAPI documentation for:

- WebSocket events
- NATS events
- Webhook events

Do not add GraphQL as the primary external API.

GraphQL may be added only for a demonstrated internal dashboard requirement.

Generate SDK clients from the OpenAPI contract where practical.

16. Authorization

Use:

- Laravel policies and gates
- Granular organization permissions
- PostgreSQL Row-Level Security
- Signed real-time channel authorization
- Tool-specific permissions
- Approval policies
- API scopes

Use a combined RBAC and ABAC model.

Authorization decisions may consider:

- Organization
- Role
- Permission
- Department
- Conversation assignment
- Customer plan
- Geographic region
- Data classification
- Tool risk level
- Approval status

For highly complex relationship-based authorization, support an OpenFGA-compatible adapter.

Do not add OpenFGA until relationship complexity justifies its operational cost.

Every authorization decision must fail closed.

17. Enterprise identity

Use standards-based identity support:

- OpenID Connect
- OAuth 2.1
- SAML 2.0
- SCIM 2.0
- TOTP MFA
- Passkey-ready WebAuthn architecture
- Recovery codes

- Session revocation

Laravel remains responsible for first-party application sessions.

Create adapters for enterprise identity providers such as:

- Microsoft Entra ID
- Okta
- Google Workspace
- Auth0
- WorkOS

Do not couple the application to one identity vendor.

18. Secrets and encryption

Use:

- Cloud KMS or HashiCorp Vault-compatible secret storage
- Envelope encryption
- Per-environment secrets
- Key rotation
- Short-lived credentials where supported
- Encrypted database fields for integration secrets

Never store plaintext:

- AI provider keys
- OAuth refresh tokens
- Shopify tokens
- CRM credentials
- Webhook secrets
- SMTP passwords
- Customer private keys

Do not place production secrets in:

- Source control
 - Docker images
 - Frontend bundles
 - Log output
 - Error reports
 - CI configuration files
-

19. Knowledge-ingestion workers

Use Python workers for:

- Web crawling
- Sitemap processing
- PDF extraction
- DOCX extraction
- Spreadsheet extraction
- HTML cleanup
- OCR when necessary
- Document classification
- Chunking
- Embedding
- Duplicate detection
- Language detection
- PII detection
- Prompt-injection scanning

Use isolated containers and strict resource limits for untrusted document processing.

Implement protections against:

- SSRF
- Private-network access
- Redirect abuse
- DNS rebinding
- Oversized files
- Decompression bombs
- Malformed documents
- Infinite crawls
- Excessive page counts
- Embedded malicious instructions
- Unsupported file formats

All ingestion jobs must be resumable and observable.

20. Shopify, WordPress, PHP, and Laravel integrations

Shopify

Use:

- Official Shopify APIs
- OAuth
- App Bridge

- Theme App Extensions
- Shopify webhooks
- GraphQL Admin API where officially recommended
- Verified customer identity
- Encrypted token storage

WordPress and WooCommerce

Use:

- Native WordPress plugin architecture
- WordPress REST APIs
- Nonces
- Capability checks
- Sanitization
- Escaping
- Prepared queries
- Signed communication with mkEngage

PHP SDK

Use:

- PSR-4
- PSR-18 HTTP client compatibility
- PSR-7 messages
- PSR-3 logging
- Composer
- Strict types

Laravel package

Use:

- Service provider
- Dependency injection
- Configuration publishing
- Queueable events
- Signed identity helpers
- Webhook verification middleware
- Laravel-compatible testing

The platform implementation language must not restrict supported customer application frameworks.

21. Mobile technology

Create native SDK foundations using:

iOS

- Swift
- Swift Package Manager
- SwiftUI-ready components
- URLSession
- Native WebSocket support
- Keychain

Android

- Kotlin
- Jetpack Compose-ready components
- Kotlin Coroutines
- OkHttp
- Encrypted local storage

Create wrappers for:

- Flutter
- React Native

Do not make the mobile product depend exclusively on a webview.

A webview may be offered as the initial lightweight integration, but native messaging, push notification, secure identity, attachment handling, and offline support must have native SDK architecture.

22. Push notifications

Use:

- Firebase Cloud Messaging for Android and web
- Apple Push Notification Service for iOS
- Web Push standard for supported browsers

Push notification tokens must be:

- Tenant-scoped
- User-scoped
- Device-scoped
- Revocable
- Expirable
- Securely stored

Do not include sensitive message content in push notifications by default.

23. Edge, CDN, and application security

Use Cloudflare or an equivalent edge provider for:

- CDN
- WAF
- DDoS protection
- Bot management
- Rate limiting
- TLS
- Custom domains
- Widget asset delivery
- Edge caching

Do not place tenant-sensitive API responses in public edge caches.

Create explicit cache-control policies for:

- Public widget assets
 - Public configuration
 - Authenticated APIs
 - Knowledge content
 - Attachments
 - Custom branding
-

24. Infrastructure and deployment

Use:

- Docker
- Docker Compose for local development
- Kubernetes for production
- Helm charts
- Terraform
- Argo CD or equivalent GitOps deployment
- Managed PostgreSQL
- Managed Redis where possible
- Managed object storage
- Managed ClickHouse where appropriate
- Managed Temporal where appropriate

Production Kubernetes must include:

- Multiple replicas
- Pod disruption budgets
- Resource requests

- Resource limits
- Horizontal Pod Autoscaling
- Readiness probes
- Liveness probes
- Startup probes
- Network policies
- Pod security settings
- Secret injection
- Rolling deployment
- Availability-zone distribution
- Backup procedures
- Disaster-recovery procedures

Do not require Kubernetes for local development.

25. Observability

Use OpenTelemetry across all services.

Collect:

- Distributed traces
- Metrics
- Structured logs
- Correlation IDs
- Organization IDs in protected internal telemetry
- Request duration
- Queue duration
- Workflow duration
- AI duration
- Retrieval duration
- Provider errors
- WebSocket connections
- Event-consumer lag
- Database latency
- ClickHouse ingestion lag

Use:

- OpenTelemetry Collector
- Prometheus
- Grafana
- Loki
- Tempo
- Sentry-compatible error tracking

Do not place message bodies, passwords, tokens, payment details, or sensitive knowledge content in telemetry.

OpenTelemetry must remain vendor-neutral so the deployment can use managed monitoring products without changing application instrumentation.

26. Testing technology

Use the appropriate test framework for every service.

Laravel

- Pest
- PHPUnit
- PHPStan
- Laravel Pint

Phoenix

- ExUnit
- Channel tests
- Presence tests
- Load tests

Python

- Pytest
- Ruff
- MyPy or Pyright
- Provider contract tests
- AI evaluation tests

Next.js and TypeScript

- Vitest
- React Testing Library
- Playwright
- ESLint
- TypeScript strict checks
- Axe

Platform testing

- k6 for load and performance
- Pact or OpenAPI contract testing
- OWASP ZAP for dynamic security testing
- Trivy for container and dependency scanning

- Gitleaks for secret scanning
- Semgrep for static security rules

The CI pipeline must fail on:

- Test failure
 - Type failure
 - Static-analysis failure
 - Migration failure
 - OpenAPI contract failure
 - Critical security vulnerability
 - Secret detection
 - Production build failure
-

27. Messaging reliability requirements

Implement:

- Transactional outbox
- Consumer inbox
- Idempotent consumers
- Dead-letter handling
- Retry policies
- Message replay
- Correlation IDs
- Causation IDs
- Per-conversation ordering
- Duplicate detection
- Delivery receipts
- Read receipts
- Offline synchronization

Define the following message lifecycle:

```
created
accepted
persisted
published
delivered
read
failed
redacted
deleted
```

A client-visible delivery confirmation must not be issued until the platform has safely accepted and persisted the message.

28. Multi-region readiness

Design for future regional deployment.

Every organization must have a configurable data region.

Prepare for regions such as:

- United States
- European Union
- United Kingdom
- India
- Australia

Region-aware components include:

- Primary database
- Object storage
- Vector data
- Conversation content
- Analytics
- AI-provider endpoint
- Backups

Do not implement active-active global writes in the initial release unless required.

Use a home-region model initially.

Global services may store only the minimum metadata required for:

- Authentication routing
- Subscription status
- Organization region resolution
- Feature entitlement

29. Data lifecycle

Implement separate storage policies for:

- Active conversation data
- Archived conversation data

- Visitor analytics
- Attachments
- Knowledge documents
- AI prompts and responses
- Audit logs
- Security events

Support:

- Configurable retention
- Legal hold
- Export
- Redaction
- Soft deletion
- Verified hard deletion
- Backup-expiration tracking
- Regional requirements

High-volume historical conversation data may be archived to object storage after the active retention window.

30. Development architecture rules

Claude must follow these rules:

1. Do not place every feature in one Laravel application.
 2. Do not create dozens of premature microservices.
 3. Keep the SaaS control plane modular.
 4. Isolate real-time messaging from administrative request processing.
 5. Isolate AI execution from synchronous user requests.
 6. Use Temporal for long-running, retryable, or approval-based operations.
 7. Use NATS JetStream for operational events.
 8. Use PostgreSQL as the transactional source of truth.
 9. Use ClickHouse for high-volume analytics.
 10. Use Redis only for ephemeral data.
 11. Use S3-compatible storage for files.
 12. Use OpenAPI and AsyncAPI contracts.
 13. Use OpenTelemetry across services.
 14. Use tenant-aware authorization at every boundary.
 15. Treat all events as at-least-once delivery.
 16. Make all consumers idempotent.
 17. Use the outbox pattern to avoid database and event-bus dual-write failures.
 18. Do not add infrastructure without a documented purpose.
 19. Create abstraction layers around replaceable external providers.
 20. Benchmark before introducing additional databases or search engines.
-

31. Required repository structure

Use a monorepo similar to:

```
mkengage/
├── apps/
│   ├── control-plane/
│   ├── dashboard/
│   ├── docs-site/
│   └── shopify-app/
├── services/
│   ├── realtime-gateway/
│   ├── ai-service/
│   ├── ingestion-worker/
│   ├── workflow-workers/
│   └── analytics-consumer/
├── packages/
│   ├── chat-widget/
│   ├── shared-contracts/
│   ├── api-client/
│   ├── react-sdk/
│   ├── vue-sdk/
│   ├── angular-sdk/
│   ├── react-native-sdk/
│   └── flutter-sdk/
├── integrations/
│   ├── php-sdk/
│   ├── laravel-package/
│   ├── wordpress-plugin/
│   ├── woocommerce/
│   └── shopify/
├── contracts/
│   ├── openapi/
│   ├── asyncapi/
│   ├── events/
│   └── webhooks/
├── infrastructure/
│   ├── docker/
│   ├── kubernetes/
│   ├── helm/
│   ├── terraform/
│   └── observability/
├── tests/
│   ├── contract/
│   └── end-to-end/
```

```
| | performance/
| | security/
| docs/
```

32. Initial deployment profile

For the first production-capable release, deploy:

- Laravel control plane
- Next.js dashboard
- Phoenix real-time gateway
- Python AI and ingestion workers
- Temporal
- NATS JetStream
- PostgreSQL with pgvector
- Redis
- ClickHouse
- S3-compatible storage
- OpenTelemetry Collector

Do not require OpenSearch in the initial deployment.

Do not require Kafka in the initial deployment.

NATS JetStream is the preferred operational event backbone for the initial architecture.

Introduce Kafka only when verified streaming throughput, ecosystem, or data-platform requirements justify the additional operational burden.

33. Claude implementation instruction

Before writing code:

1. Inspect the repository.
2. Produce an architecture decision record for every major technology.
3. Create service boundaries.
4. Define OpenAPI contracts.
5. Define AsyncAPI contracts.
6. Define event schemas.
7. Define tenant-isolation rules.
8. Define message-ordering rules.
9. Define failure and retry behavior.
10. Define the minimum production deployment.

Create these files first:

```
docs/architecture/ADR-001-platform-architecture.md
docs/architecture/ADR-002-realtime-gateway.md
docs/architecture/ADR-003-ai-service.md
docs/architecture/ADR-004-workflow-engine.md
docs/architecture/ADR-005-event-backbone.md
docs/architecture/ADR-006-data-platform.md
docs/architecture/ADR-007-multi-tenancy.md
docs/architecture/ADR-008-observability.md
docs/architecture/ADR-009-security-boundaries.md
docs/architecture/ADR-010-deployment-model.md
```

Each architecture decision record must contain:

- Context
- Decision
- Alternatives considered
- Advantages
- Disadvantages
- Security implications
- Operational implications
- Cost implications
- Migration path
- Reversal strategy

After completing the architecture decisions, continue directly with Phase 1 implementation.

Do not stop after writing architecture documents.

Do not claim a service is production-ready until its tests, security checks, health checks, observability, failure handling, and deployment configuration have been verified.